CSE 331/531 Project 3:

32-bit Single Cycle MIPS Design

Deadline: 19/12/2024 Third Project Exam

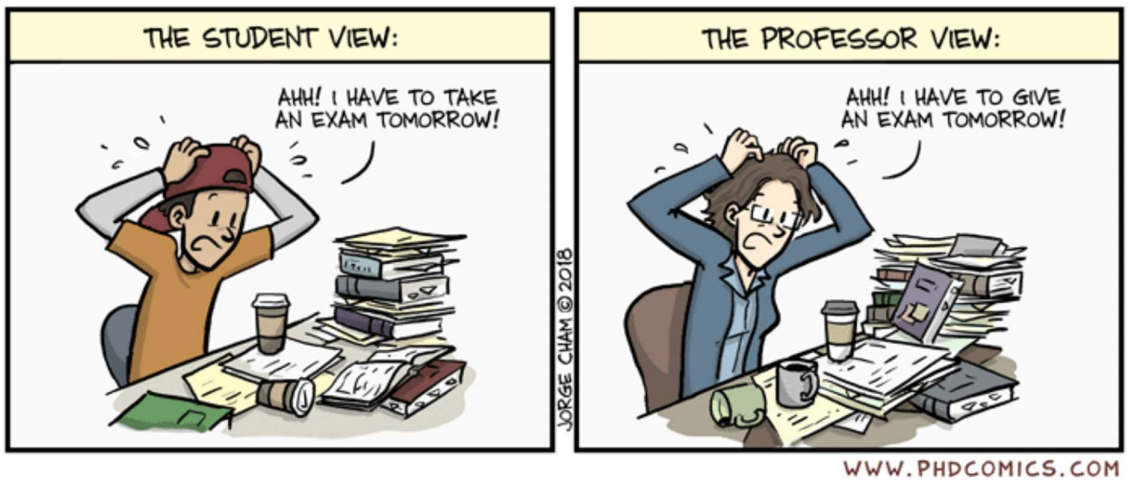
1. Project Overview

A single-cycle MIPS processor executes one instruction per clock cycle, completing all the steps—fetch, decode, execute, memory access, and write back—within a single cycle. Let's explore each module of the design, its purpose, functionality, and how they interact. We'll also discuss the role of the testbench in verifying the processor's functionality.

List of Supported Instructions:

1. **Arithmetic:** add, addi, sub
2. **Logical Operations:** and, andi, or, ori
3. **Comparison:** slt
4. **Memory Access:** lw, sw
5. **Branching:** beq, bne
6. **Jumping:** j

These instructions cover fundamental operations, allowing the processor to execute basic programs, handle branching, and access memory effectively. Together, they provide a strong foundation for understanding MIPS architecture and verifying the processor's functionality using a testbench.



2. Datapath Module

Purpose:
The datapath carries out the core operations of the processor by moving data between various functional units, performing calculations, and storing results.

Key Components and Their Roles:

- **Program Counter (PC):**
  The PC holds the address of the current instruction. After each instruction fetch, it typically increments by 4 to point to the next instruction. For jumps or branches, the PC is updated with a different value to change the execution flow.
- **Instruction Fetch Unit:**
  - Retrieves the instruction from instruction memory using the address provided by the PC.
  - Passes the fetched instruction to the control unit for decoding and to other datapath components for processing.
- **Register File:**
  - Contains 32 general-purpose registers ($0 to $31). Each register is 32 bits wide.
  - Provides two read ports (for operands) and one write port (for results).
  - Reads are performed asynchronously for speed, while writes occur synchronously on the clock edge.
- **Arithmetic Logic Unit (ALU):**
  - Performs arithmetic operations like addition and subtraction, as well as logical operations like AND, OR, and set-on-less-than.
  - Takes two inputs (operands), processes them based on control signals, and produces an output.
  - Generates a zero flag, which is critical for conditional branch instructions (beq and bne).
- **Sign-Extender:**
  - Extends 16-bit immediate values to 32 bits, preserving the sign (important for operations like addi).
  - Ensures consistent handling of immediate values in the datapath.
- **Multiplexers (MUX):**
  - Route data based on control signals. For example:
    - ⇨ Select between a register value or an immediate value as an ALU input.
    - ⇨ Choose the destination register for write-back operations.

- ⇨ Decide whether to increment the PC or branch to a different address.

Data Flow:

- The instruction is fetched based on the PC value.
- The fetched instruction is decoded to determine which registers to read and what operation to perform.
- The ALU processes the operands, and results may be written back to registers or memory.
- The PC is updated to fetch the next instruction, taking into account possible jumps or branches.

3. Control Unit

Purpose:
The control unit generates signals that guide the operations of the datapath. It translates the fetched instruction into control signals that dictate how the datapath components interact.

Key Responsibilities:

- **Instruction Decoding:**
  The control unit examines the opcode (and the function field for R-type instructions) to determine the type of instruction (R-type, I-type, or J-type) and what operations are required.
- **Signal Generation:**
  Produces signals such as:
  - **RegDst:** Chooses the destination register.
  - **ALUSrc:** Determines whether the ALU's second operand comes from a register or an immediate value.
  - **MemRead / MemWrite:** Control access to data memory.
  - **MemToReg:** Selects the source of data to write back to the register file.
  - **Branch / Jump:** Alters the PC if the instruction is a branch or jump.
  - **ALUOp:** Specifies the operation for the ALU (e.g., add, subtract, logical AND).

Example Behavior:

- For an add instruction (R-type), the control unit sets signals to use registers as inputs and to write the result back to a register.
- For a lw instruction (I-type), it sets signals to read from data memory and write the result to a register.

4. Instruction Memory

Purpose:
Stores the machine code (binary instructions) that the processor executes.

Characteristics:

- **Read-Only:** Typically read-only during execution.
- **Addressable:** Each instruction has a unique address, and instructions are fetched sequentially unless altered by a jump or branch.

How It Works:

- The PC provides an address to instruction memory.
- The corresponding instruction is fetched and passed to the control unit and datapath.

Initialization:
The memory is preloaded with the program's instructions, often using an initial block in simulation or loaded from an external file in hardware.

5. Data Memory

Purpose:
Holds data that the program needs during execution, especially for lw (load word) and sw (store word) instructions.

Characteristics:

- **Read/Write:** Supports both reading and writing data.
- **Word-Aligned:** Accesses are typically aligned to 32-bit words.

Functionality:

- During a lw instruction, data is read from the memory and loaded into a register.
- During a sw instruction, data from a register is stored in memory.

6. ALU (Arithmetic Logic Unit)

Purpose:
Performs all arithmetic and logical operations in the processor.

Operations Supported:

- Arithmetic: Add, subtract.
- Logical: AND, OR.
- Comparison: Set-on-less-than (SLT).

Control Signals:
The ALU operation is determined by signals from the control unit. For example, an add operation requires different control signals than an and operation.

7. Registers (Register File)

Purpose:
Stores temporary data used by the processor. Registers provide fast access to operands for the ALU and hold results between operations.

Features:

- 32 general-purpose registers.
- Dual read ports and one write port.
- $0 is always zero by convention.

Usage:
Most instructions operate on registers, retrieving values to perform operations and storing results back.

8. Testbench

Purpose:
The testbench is a simulation environment used to verify that the processor works correctly. It provides inputs (instructions and data) and checks that the outputs match expected values.

Key Components:

- **Clock Generation:** Provides the clock signal for synchronous components.
- **Reset Logic:** Ensures the processor starts from a known state.
- **Instruction Loading:** Loads a set of instructions into instruction memory.
- **Monitor and Assertions:** Checks the processor's outputs, such as register values and memory contents, after executing instructions.

Testing Strategy:

1. **Program Execution:** Load a series of instructions that cover all supported operations (arithmetic, memory access, branching).
2. **Check Results:** After execution, verify the values in the registers and memory against expected outcomes.
3. **Edge Cases:** Test scenarios like arithmetic overflows, invalid memory accesses, and branch conditions.